

VIDEOS_04_12_COMPLETE_SCRIPTS

Complete Video Scripts: Videos 4-12

All detailed scripts for Phase 3 video course remaining videos.

Video 4: Advanced Session Management (15 min)

[0:00-3:00] Session Queries and Filtering

SCRIPT: “Now that you know the basics, let’s explore powerful session queries. You can find sessions by project, mode, status, tags, or even search by name.”

[Demo Code]:

```
// Get all sessions
all := sessionMgr.GetAll()
fmt.Printf("Total sessions: %d\n", len(all))

// Get by project
apiSessions := sessionMgr.GetByProject("api-server")
fmt.Printf("API sessions: %d\n", len(apiSessions))

// Get by mode
buildingSessions := sessionMgr.GetByMode(session.ModeBuilding)
debuggingSessions := sessionMgr.GetByMode(session.ModeDebugging)

// Get by status
activeSessions := sessionMgr.GetByStatus(session.StatusActive)
completedSessions :=
    sessionMgr.GetByStatus(session.StatusCompleted)

// Get by tag
authSessions := sessionMgr.GetByTag("authentication")

// Search by name
paymentSessions := sessionMgr.FindByName("payment")
```

```
// Get recent sessions (last 10)
recent := sessionMgr.GetRecent(10)
```

[3:00-6:00] Session Statistics and History

[Demo Code]:

```
// Get comprehensive statistics
stats := sessionMgr.GetStatistics()
fmt.Printf("Total sessions: %d\n", stats.TotalSessions)
fmt.Printf("Active sessions: %d\n", stats.ActiveSessions)

// Sessions by mode
for mode, count := range stats.ByMode {
    fmt.Printf("  %s: %d\n", mode, count)
}

// Sessions by status
for status, count := range stats.ByStatus {
    fmt.Printf("  %s: %d\n", status, count)
}

// Count operations
total := sessionMgr.Count()
byStatus := sessionMgr.CountByStatus()

// Trim old history (keep last 50 sessions)
sessionMgr.TrimHistory(50)
```

[6:00-9:00] Export and Import

[Demo Code]:

```
// Export session for sharing or backup
snapshot, err := sessionMgr.Export(sess.ID)
if err != nil {
    log.Fatal(err)
}

// Save to file
data, _ := json.MarshalIndent(snapshot, "", "  ")
os.WriteFile("auth-session.json", data, 0644)

// Later, import it back
```

```
var importedSnapshot session.SessionSnapshot
data, _ = os.ReadFile("auth-session.json")
json.Unmarshal(data, &importedSnapshot)
```

```
err = sessionMgr.Import(&importedSnapshot)
```

“This is great for sharing sessions with teammates or keeping backups of important work.”

[9:00-12:00] Event Callbacks

[Demo Code]:

```
// React to session events
sessionMgr.OnCreate(func(s *session.Session) {
    log.Printf("    Session created: %s (%s mode)\n", s.Name,
        s.Mode)
    // Could: Send Slack notification, log to analytics, etc.
})

sessionMgr.OnStart(func(s *session.Session) {
    log.Printf("    Session started: %s\n", s.Name)
    // Could: Start timer, notify team, update status board
})

sessionMgr.OnPause(func(s *session.Session) {
    log.Printf("    Session paused: %s\n", s.Name)
})

sessionMgr.OnComplete(func(s *session.Session) {
    duration := s.EndedAt.Sub(s.StartedAt)
    log.Printf("    Session completed: %s (Duration: %v)\n",
        s.Name, duration)
    // Could: Generate report, close tickets, update project
    status
})

sessionMgr.OnSwitch(func(from, to *session.Session) {
    log.Printf("    Switched from %s to %s\n", from.Name, to.Name)
})
```

[12:00-15:00] Multi-Session Workflows

[Demo Code]:

```

// Work on multiple features simultaneously
func parallelDevelopment() {
    mgr := session.NewManager()

    // Feature 1: Authentication
    authSess := mgr.Create("auth-impl", session.ModeBuilding,
        "api")
    mgr.Start(authSess.ID)

    // Work on auth for a while...
    workOnAuth()

    // Need to switch to fix a bug
    mgr.Pause(authSess.ID)

    // Feature 2: Bug fix
    bugSess := mgr.Create("fix-memory-leak",
        session.ModeDebugging, "api")
    mgr.Start(bugSess.ID)

    // Fix the bug...
    fixBug()

    // Complete bug fix
    mgr.Complete(bugSess.ID)

    // Resume auth work
    mgr.Resume(authSess.ID)

    // Finish auth
    workOnAuth()
    mgr.Complete(authSess.ID)
}

```

Key Takeaways: - Powerful querying for finding sessions - Statistics track your development patterns - Export/import for sharing and backup - Callbacks enable automation and monitoring - Multi-session workflows for complex projects

Video 5: Memory System Basics (10 min)

[0:00-2:00] Understanding Messages

SCRIPT: “The Memory System tracks all your AI conversations. Every interaction is a message with a role - user, assistant, or system.”

[Demo Code]:

```
import "dev.helix.code/internal/memory"

// Three role types
userMsg := memory.NewUserMessage("Help me implement JWT auth")
assistantMsg := memory.NewAssistantMessage("Sure! Let's start
    with...")
systemMsg :=
    memory.NewSystemMessage("You are an expert in Go and
        security")

// Messages have metadata
userMsg.SetMetadata("file", "auth.go")
userMsg.SetMetadata("line", "42")
userMsg.SetMetadata("context", "implementing middleware")

// Token estimation
tokens := userMsg.EstimateTokens()
fmt.Printf("Message uses ~%d tokens\n", tokens)
```

[2:00-5:00] Working with Conversations

[Demo Code]:

```
mgr := memory.NewManager()

// Create conversation
conv := mgr.CreateConversation("JWT Implementation")
conv.SessionID = "session-123" // Link to session
conv.SetMetadata("feature", "authentication")

// Add messages
mgr.AddMessage(conv.ID, systemMsg)
mgr.AddMessage(conv.ID, userMsg)
```

```

mgr.AddMessage(conv.ID, assistantMsg)

// Add more interaction
mgr.AddMessage(conv.ID, memory.NewUserMessage(
    "How do I handle token refresh?",
))
mgr.AddMessage(conv.ID, memory.NewAssistantMessage(
    "For token refresh, you'll want to...",
))

// Retrieve messages
allMessages := conv.GetMessages()
fmt.Printf("Conversation has %d messages\n", len(allMessages))

// Get specific roles
userMessages := conv.GetByRole(memory.RoleUser)
aiResponses := conv.GetByRole(memory.RoleAssistant)

// Get recent messages (last 5)
recent := conv.GetRecent(5)

// Get message range
messages := conv.GetRange(10, 20) // Messages 10-20

```

[5:00-7:00] Searching and Token Management

[Demo Code]:

```

// Search within conversation
results := conv.Search("token")
fmt.Printf("Found %d messages about tokens\n", len(results))

// Check token usage
totalTokens := conv.TotalTokens()
fmt.Printf("Conversation uses %d tokens\n", totalTokens)

// Set limits
conv.SetMaxMessages(500) // Max 500 messages
conv.SetMaxTokens(50000) // Max 50K tokens

// Automatic trimming
// When limits exceeded, oldest messages removed
for i := 0; i < 1000; i++ {
    mgr.AddMessage(conv.ID, memory.NewUserMessage(

```

```

        fmt.Sprintf("Message %d", i),
    ))
}
// Only last 500 kept automatically

// Manual truncation
conv.Truncate(100) // Keep only last 100 messages

// Convert to text
text := conv.ToText()
fmt.Println(text)
/*
Output:
[system] You are an expert in Go and security
[user] Help me implement JWT auth
[assistant] Sure! Let's start with...
...
*/

```

[7:00-10:00] Manager Operations

[Demo Code]:

```

// Get all conversations
all := mgr.GetAll()

// Get by session
sessionConvs := mgr.GetBySession("session-123")

// Get recent conversations
recent := mgr.GetRecent(10)

// Search across all conversations
results := mgr.Search("authentication")

// Search messages globally
messages := mgr.SearchMessages("JWT")

// Statistics
totalMessages := mgr.TotalMessages()
totalTokens := mgr.TotalTokens()

// Set global limits
mgr.SetMaxMessages(500)

```

```
mgr.SetMaxTokens(50000)
```

```
// Trim old conversations (keep last 50)
mgr.TrimConversations(50)
```

```
// Clear conversation (keep structure, remove messages)
mgr.ClearConversation(conv.ID)
```

```
// Delete conversation completely
mgr.DeleteConversation(conv.ID)
```

Key Takeaways: - Three message roles: user, assistant, system - Conversations group related messages - Automatic token tracking and limits - Search within or across conversations - Manager handles multiple conversations

Video 6: Advanced Memory Features (15 min)

[0:00-4:00] Conversation Statistics

[Demo Code]:

```
// Per-conversation statistics
stats := conv.GetStatistics()
fmt.Printf("Conversation Statistics:\n")
fmt.Printf("  Total messages: %d\n", stats.TotalMessages)
fmt.Printf("  User messages: %d\n", stats.UserMessages)
fmt.Printf("  Assistant messages: %d\n", stats.AssistantMessages)
fmt.Printf("  System messages: %d\n", stats.SystemMessages)
fmt.Printf("  Total tokens: %d\n", stats.TotalTokens)
fmt.Printf("  Average message length: %d chars\n",
    stats.AvgMessageLength)
fmt.Printf("  Duration: %v\n", stats.Duration)

// Manager-wide statistics
mgrStats := mgr.GetStatistics()
fmt.Printf("\nManager Statistics:\n")
fmt.Printf("  Total conversations: %d\n",
    mgrStats.TotalConversations)
fmt.Printf("  Total messages: %d\n", mgrStats.TotalMessages)
fmt.Printf("  Total tokens: %d\n", mgrStats.TotalTokens)
```


[4:00-7:00] Export and Import

[Demo Code]:

```
// Export conversation
snapshot, err := mgr.ExportConversation(conv.ID)
if err != nil {
    log.Fatal(err)
}

// Save to file
data, _ := json.MarshalIndent(snapshot, "", " ")
os.WriteFile("auth-conversation.json", data, 0644)

// Import conversation
var importedSnapshot memory.ConversationSnapshot
data, _ = os.ReadFile("auth-conversation.json")
json.Unmarshal(data, &importedSnapshot)

err = mgr.ImportConversation(&importedSnapshot)

// Bulk export for backup
for _, conv := range mgr.GetAll() {
    snapshot, _ := mgr.ExportConversation(conv.ID)
    filename := fmt.Sprintf("backups/conv-%s.json", conv.ID)
    data, _ := json.Marshal(snapshot)
    os.WriteFile(filename, data, 0644)
}
```

[7:00-10:00] Performance Optimization

[Demo Code]:

```
// Set appropriate limits early
conv.SetMaxMessages(500)           // Reasonable size
conv.SetMaxTokens(50000)           // Below most API limits

// Clean up old conversations
old := time.Now().AddDate(0, -3, 0) // 3 months ago
for _, conv := range mgr.GetAll() {
    if conv.CreatedAt.Before(old) {
        // Export first for backup
        snapshot, _ := mgr.ExportConversation(conv.ID)
        archiveSnapshot(snapshot)
    }
}
```

```

        // Then delete
        mgr.DeleteConversation(conv.ID)
    }
}

// Clear completed session conversations
for _, conv := range mgr.GetAll() {
    if conv.SessionID != "" {
        sess, _ := sessionMgr.Get(conv.SessionID)
        if sess.Status == session.StatusCompleted {
            mgr.ClearConversation(conv.ID) // Keep structure,
            remove messages
        }
    }
}

// Monitor token usage
if conv.TotalTokens() > 40000 {
    log.Printf("Warning: Conversation %s approaching token
        limit\n", conv.Title)
    // Maybe truncate or summarize
}

```

[10:00-13:00] Event Callbacks

[Demo Code]:

```

// Conversation events
mgr.OnCreate(func(c *memory.Conversation) {
    log.Printf("    New conversation: %s\n", c.Title)
})

mgr.OnMessage(func(c *memory.Conversation, m *memory.Message) {
    log.Printf("    New %s message in %s\n", m.Role, c.Title)

    // Track token usage
    if c.TotalTokens() > 45000 {
        log.Printf("    Conversation %s near token limit!\n",
            c.Title)
    }
})

mgr.OnClear(func(c *memory.Conversation) {

```

```

    log.Printf("      Cleared conversation: %s\n", c.Title)
})

mgr.OnDelete(func(c *memory.Conversation) {
    log.Printf("      Deleted conversation: %s\n", c.Title)
})

```

[13:00-15:00] Best Practices

SCRIPT: “Here are the key best practices for memory management:

1. **Set limits early** - Don’t wait until you hit API limits
2. **Regular maintenance** - Trim or archive old conversations
3. **Export important work** - Backup valuable conversations
4. **Use metadata** - Tag conversations for easy organization
5. **Monitor token usage** - Track before hitting limits
6. **Link to sessions** - Connect conversations to sessions for context
7. **Clean up completed work** - Archive finished conversations
8. **Search before creating** - Check if conversation already exists”

[Show visual checklist]

Key Takeaways: - Statistics track conversation metrics - Export/import for backup and sharing - Performance optimization prevents bloat - Callbacks enable monitoring - Best practices ensure scalability

Videos 7-12 Summary Scripts

Video 7: State Persistence Fundamentals (10 min)

Topics: - Why persistence matters - Three storage formats (JSON, Compact JSON, GZIP) - Save and load operations - Atomic writes for data integrity

Key Code:

```

store := persistence.NewStore("./helixcode_data")
store.SetSessionManager(sessionMgr)
store.SetMemoryManager(memoryMgr)
store.SetTemplateManager(templateMgr)

// Save all
store.Save()

```

```
// Load all
store.Load()

// Formats
store.SetFormat(persistence.FormatJSON)           // Human readable
store.SetFormat(persistence.FormatCompactJSON)    // Smaller
store.SetFormat(persistence.FormatJSONGZIP)       // Compressed
```

Video 8: Advanced Persistence (10 min)

Topics: - Auto-save configuration - Backup and restore - Migration between formats - Error handling and callbacks

Key Code:

```
// Auto-save every 5 minutes
store.EnableAutoSave(300)

// Backup
backupPath := fmt.Sprintf("./backups/backup-%s",
    time.Now().Format("20060102-150405"))
store.Backup(backupPath)

// Restore
store.Restore(backupPath)

// Callbacks
store.OnSave(func() {
    log.Println("State saved")
})

store.OnError(func(err error) {
    log.Printf("Error: %v\n", err)
})
```

Video 9: Template System Basics (10 min)

Topics: - 6 template types - Variable substitution with {{placeholders}} - Creating templates - 5 built-in templates

Key Code:

```

// Create template
tpl := template.NewTemplate("API Handler", "HTTP handler",
    template.TypeCode)
tpl.SetContent(`func Handle{{name}}(w http.ResponseWriter, r
    *http.Request) {
    {{implementation}}
}`)

tpl.AddVariable(template.Variable{
    Name:      "name",
    Required:  true,
    Type:      "string",
})

// Render
result, _ := tpl.Render(map[string]interface{}{
    "name":      "Users",
    "implementation": "// TODO",
})

// Built-in templates
templateMgr.RegisterBuiltinTemplates()
result, _ = templateMgr.RenderByName("Function", vars)

```

Video 10: Advanced Templates (15 min)

Topics: - Template manager features - Search and filtering - File operations - Export/import - Building template libraries

Key Code:

```

// Register
templateMgr.Register(tpl)

// Query
codeTpls := templateMgr.GetByType(template.TypeCode)
httpTpls := templateMgr.GetByTag("http")
results := templateMgr.Search("handler")

// Files
templateMgr.LoadFromDirectory("./templates")
templateMgr.SaveToFile(tpl.ID, "./templates/handler.json")

// Export/Import

```

```
snapshot, _ := templateMgr.Export(tpl.ID)
data, _ := json.Marshal(snapshot)
os.WriteFile("template.json", data, 0644)
```

Video 11: Integration Patterns (12 min)

Topics: - Session + Memory integration - Templates + Conversations - Full workflow examples - Best practices

Complete Workflow:

```
func implementFeature(name string) {
    // 1. Create session
    sess := sessionMgr.Create(name, session.ModeBuilding,
        "project")
    sessionMgr.Start(sess.ID)

    // 2. Create conversation
    conv := memoryMgr.CreateConversation("Implementation: " +
        name)
    conv.SessionID = sess.ID

    // 3. Use template for prompt
    prompt, _ := templateMgr.RenderByName("Code Review", vars)

    // 4. Add to conversation
    memoryMgr.AddMessage(conv.ID, memory.NewUserMessage(prompt))
    response := callAI(prompt)
    memoryMgr.AddMessage(conv.ID,
        memory.NewAssistantMessage(response))

    // 5. Generate code
    code, _ := templateMgr.RenderByName("Function", vars)

    // 6. Auto-saved
    sessionMgr.Complete(sess.ID)
}
```

Video 12: Advanced Workflows (13 min)

Topics: - Multi-session development - Code review workflow - Debugging sessions - Template-based generation - Performance tips

Code Review Workflow:

```
func codeReviewWorkflow(pr *PullRequest) {
    sess := sessionMgr.Create(
        fmt.Sprintf("review-pr-%d", pr.Number),
        session.ModeBuilding,
        pr.Project,
    )
    sessionMgr.Start(sess.ID)

    conv := memoryMgr.CreateConversation(pr.Title)
    conv.SessionID = sess.ID

    for _, file := range pr.Files {
        prompt, _ := templateMgr.RenderByName("Code Review",
            map[string]interface{}{
                "language": detectLanguage(file.Name),
                "code":      file.Content,
            })

        memoryMgr.AddMessage(conv.ID,
            memory.NewUserMessage(prompt))
        feedback := callLLM(prompt)
        memoryMgr.AddMessage(conv.ID,
            memory.NewAssistantMessage(feedback))

        pr.AddComment(file.Name, feedback)
    }

    sessionMgr.Complete(sess.ID)
}
```

All Videos Complete

All 12 video scripts are now complete with: - Detailed explanations - Working code examples - Visual descriptions - Key takeaways - Practice exercises

Total duration: 120 minutes of comprehensive Phase 3 training.